26. Handle CAP Theorem Trade-offs

Key Terms:

Term	Full Name	What it does
CAP	Consistency, Availability, Partition tolerance	Theorem: distributed systems can only guarantee 2 of 3
CP	Consistency + Partition tolerance	Sacrifice availability for data accuracy
AP	Availability + Partition tolerance	Sacrifice consistency for uptime
PACELC	Partition-Availability-Consistency Else Latency-Consistency	Extension of CAP for normal operation
Split-brain	Network partition scenario	System divides into isolated groups
Bounded Staleness	Consistency level	Data within N seconds/versions of latest

Problem Statement

Understand and apply CAP theorem when designing distributed systems, making conscious trade-offs between consistency, availability, and partition tolerance.

Clarifying Questions to Ask

1. **Data type**: What kind of data? (Financial, social, inventory?)
2. **Users**: How do users interact? (Read-heavy, write-heavy?)
3. **Network**: Single datacenter or multi-region?
4. **Failure tolerance**: Which failures must the system survive?
5. **Business impact**: Cost of inconsistency vs. unavailability?
6. **Recovery**: How to recover from split-brain scenarios?
7. **SLAs**: What availability/consistency SLAs are required?

API Design

Consistency Configuration APIs

Method	Endpoint	Description
`GET`	`/api/v1/config/consistency`	Get current consistency settings
`PUT`	`/api/v1/config/consistency`	Update consistency level
`GET`	`/api/v1/partitions/status`	Check partition health

Data Access APIs (consistency-aware)

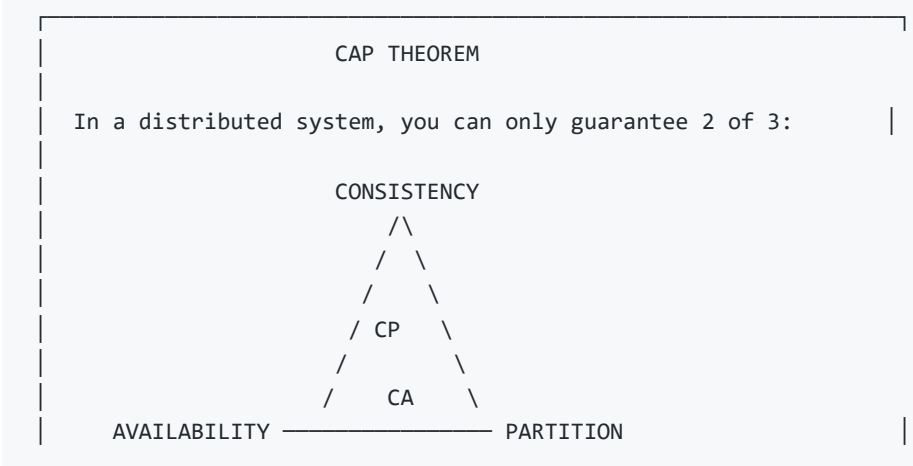
```
// AP System (Available + Partition-tolerant)
GET /api/v1/data/{key}
Headers:
  X-Consistency: eventual
  X-Max-Staleness: 5s

// CP System (Consistent + Partition-tolerant)
GET /api/v1/data/{key}
Headers:
  X-Consistency: strong
  X-Read-Quorum: majority
```

Response Headers (consistency metadata)

```
X-Data-Version: 42
X-Last-Modified: 2025-03-15T10:00:00Z
X-Source-Region: us-east
X-Replication-Lag-Ms: 150
```

CAP Theorem Explained



```
│                    \    AP    /    TOLERANCE                    │
│                     \        /                                   │
│                      \      /                                    │
│                       \    /                                     │
│                        \  /                                      │
│                         \/                                       │
│                                                                  │
│  CA: Not practical (network partitions happen)                   │
│  CP: Sacrifice availability for consistency                      │
│  AP: Sacrifice consistency for availability                      │
└──────────────────────────────────────────────────────────────────┘
```

Trade-off Scenarios

```
SCENARIO 1: BANKING SYSTEM (Choose CP)

┌──────────────────────────────────────────────────────────────────┐
│  Requirement: Account balance must be accurate                   │
│  Trade-off: Reject transactions during partition                 │
│                                                                  │
│  During network partition:                                       │
│    - Transfer request arrives                                    │
│    - Cannot verify balance with other nodes                      │
│    - Return error: "Service temporarily unavailable"            │
│    - Better to fail than allow overdraft                         │
│                                                                  │
│  Technologies: PostgreSQL with synchronous replication           │
│                ZooKeeper, etcd                                   │
└──────────────────────────────────────────────────────────────────┘

SCENARIO 2: SOCIAL MEDIA FEED (Choose AP)

┌──────────────────────────────────────────────────────────────────┐
│  Requirement: Users can always view and post                     │
│  Trade-off: May see slightly stale feed                          │
│                                                                  │
│  During network partition:                                       │
│    - User posts update                                           │
│    - Stored locally, queued for replication                      │
│    - Other users may not see it immediately                      │
│    - Eventually consistent when partition heals                  │
│                                                                  │
│  Technologies: Cassandra, DynamoDB, CouchDB                      │
└──────────────────────────────────────────────────────────────────┘

SCENARIO 3: SHOPPING CART (Choose AP with reconciliation)

┌──────────────────────────────────────────────────────────────────┐
│  Requirement: Users can always add items                         │
│  Trade-off: Merge carts when partition heals                     │
│                                                                  │
│  During partition:                                               │
│    - User adds "apple" on device A                               │
│    - User adds "banana" on device B                              │
│    - Both writes succeed locally                                 │
│                                                                  │
```

```
│  After partition heals:                                          │
│    - Merge: cart = ["apple", "banana"]                           │
│    - No data lost, both actions preserved                        │
│                                                                  │
│  Conflict resolution: Union (for add), LWW (for remove)          │
└──────────────────────────────────────────────────────────────────┘
```

PACELC Extension

```
┌──────────────────────────────────────────────────────────────────┐
│  PACELC: Extends CAP for normal operation                        │
│                                                                  │
│  If Partition:                                                   │
│    Choose Availability or Consistency (CAP)                      │
│  Else (normal operation):                                        │
│    Choose Latency or Consistency                                 │
│                                                                  │
│  Examples:                                                       │
│    PA/EL (DynamoDB): Available during partition, low latency     │
│    PC/EC (Traditional SQL): Consistent always, higher latency    │
│    PA/EC (Cosmos DB): Available in partition, consistent else    │
└──────────────────────────────────────────────────────────────────┘
```

Consistency Levels (Cosmos DB Example)

```
┌──────────────────────────────────────────────────────────────────┐
│  STRONG: Read sees latest write (highest latency)                │
│    ├── Use for: Financial transactions                           │
│    └── Cost: Higher latency, lower throughput                    │
│                                                                  │
│  BOUNDED STALENESS: Read within N seconds/versions of latest     │
│    ├── Use for: Leaderboards, activity feeds                     │
│    └── Cost: Moderate latency                                    │
│                                                                  │
│  SESSION: Read-your-own-writes within session                    │
│    ├── Use for: User profiles, shopping carts                    │
│    └── Cost: Low latency                                         │
│                                                                  │
│  EVENTUAL: No ordering guarantee (lowest latency)                │
│    ├── Use for: Like counts, view counts                         │
│    └── Cost: May see stale data                                  │
└──────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Choose CP	Choose AP

Banking	✓ (accuracy critical)	
Social media		✓ (availability > consistency)
Inventory	✓ (overselling bad)	Or AP with reservation
DNS		✓ (must resolve)
Configuration	✓ (all nodes same config)	
Session data		✓ (user always logged in)

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Cosmos DB	Tunable consistency with PACELC support	Tech Guide
PostgreSQL	CP system with synchronous replication	Tech Guide
Cassandra	AP system for high availability	Tech Guide

How to Explain It Well

> **Opening**: "CAP theorem states we can't have all three: consistency, availability, and partition tolerance. I choose based on business requirements."

> **Key talking points**:
>
> - "Network partitions are inevitable - CA isn't practical"
> - "Banking chooses CP: reject transactions vs. risk inconsistency"
> - "Social media chooses AP: stale feed is better than no feed"
> - "PACELC extends CAP: even without partition, latency vs. consistency trade-off exists"
> - "Cosmos DB lets you choose per-operation: strong for critical, eventual for counters"
